Muhurat AI (मुहूर्त AI)

Full-Stack Agentic AI Engineering • High & Low Level System Design • FAANG Interview Master Guide

NEXT.JS 16 | GOOGLE GEMINI 3.6 FLASH | MASTRA AI MEMORY | DESCOPE OUTBOUND APPS | POSTGRESQL & LIBSQL | MODEL CONTEXT PROTOCOL (MCP)

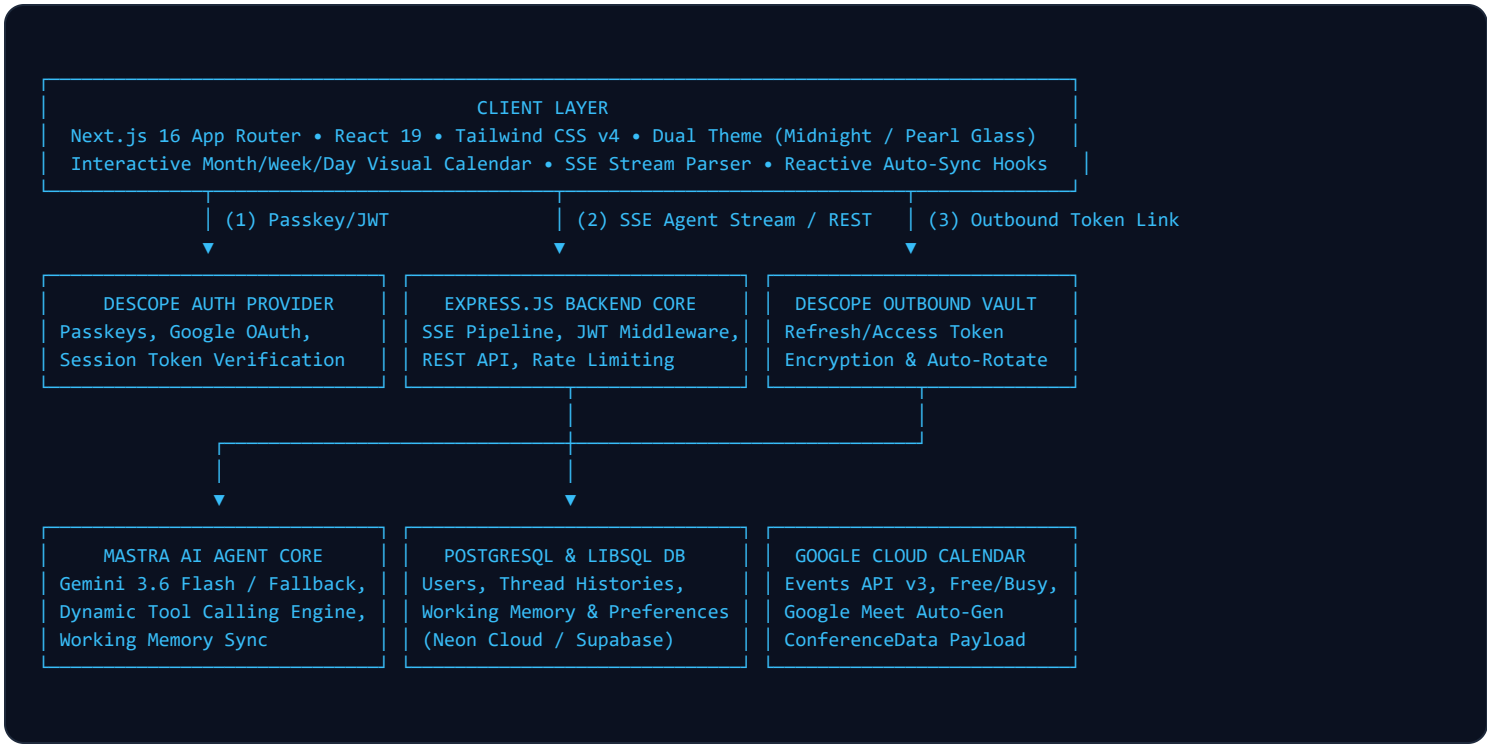
1. Executive Summary & Product Vision

Muhurat AI is an autonomous calendar intelligence platform inspired by the ancient Vedic philosophy of *Muhurat* (identifying the optimal, conflict-free, and auspicious time window for actions). Built as an enterprise-grade agentic assistant, it blends conversational reasoning with deterministic Google Calendar orchestration.

Unlike standard calendar chatbots that merely parse text with static regex, Muhurat AI operates as a **multi-step autonomous agent loop**: it reasons through free/busy matrices, negotiates overlapping slots, generates instantaneous Google Meet conference links, commits changes to Google Cloud, and remembers multi-turn scheduling habits across conversations via persistent working memory.

2. High-Level Design (HLD)

The system is architected around a modern decoupled topology ensuring real-time bidirectional synchronization, zero-latency streaming, and vault-grade OAuth security:



Key Architectural Components:

Component	Technology	Core Responsibility
Frontend Client	Next.js 16, React 19, Tailwind v4	Responsive glassmorphic UI, Split View dashboard, real-time SSE stream renderer, month/week/day interactive Google Calendar view.
Identity & Token Vault	Descope Next.js & Node SDK	Stateless JWT session authentication and encrypted storage/rotation of third-party Google OAuth2 tokens.
Backend Orchestrator	Node.js, Express.js, TypeScript	Validates session claims, proxies SSE agent streams, runs database schema migrations, mounts MCP tools.
Agent Engine	Mastra AI Core & Gemini 3.6 Flash	Multi-provider LLM orchestration, structured tool execution loops, context maintenance, and semantic validation.
Persistent Memory	Mastra Memory + LibSQL / Postgres	Cross-session retention of user scheduling constraints, preferred meeting durations, working hours, and favorite attendees.
MCP Integration	Model Context Protocol Express	Exposes calendar management tools over standardized MCP endpoints to external IDEs (Antigravity, Cursor, Claude).

3. Low-Level Design (LLD) & Data Modeling

3.1 Database Schema (PostgreSQL)

```
-- 1. Users Table (Stores user entity mapped to Descope Auth ID)
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE IF NOT EXISTS users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    auth_user_id TEXT UNIQUE NOT NULL,
    email TEXT,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 2. Connections Table (Tracks OAuth connection state)
CREATE TABLE IF NOT EXISTS connections (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    provider TEXT NOT NULL,
    status TEXT NOT NULL DEFAULT 'connected',
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE (user_id, provider)
);
```

3.2 Agent Tool Definitions & Schema Contracts

All tools are strongly typed using **Zod** schemas. When the agent receives a natural language instruction, Gemini 3.6 Flash evaluates the JSON schema contract and invokes the tool deterministically:

- `createMeetingTool`: Accepts `title`, `startTime` (ISO 8601), `endTime`, `attendees` (array of emails), and `description`. Injects `conferenceDataVersion: 1` with `createRequest: { requestId, conferenceSolutionKey: { type: "hangoutsMeet" } }` to create a live Google Meet room.
- `checkCalendarBusyTool`: Queries Google Calendar Free/Busy API (`calendar.freebusy.query`) over a 24-hour range to detect conflicts before booking.
- `listUpcomingMeetingsTool`: Retrieves structured agenda for a time window (e.g. today, next 7 days).
- `rescheduleMeetingTool`: Modifies existing calendar event instances while maintaining attendee notifications.
- `cancelMeetingTool`: Deletes confirmed calendar events and triggers cancellation emails to attendees.

3.3 Real-Time Server-Sent Events (SSE) Protocol

```
// SSE Chunk Protocol emitted from /api/agent/stream:
event: started    -> data: {"threadId": "uuid-v4"}
event: progress   -> data: {"message": "Checking calendar availability for tomorrow morning..."}
event: token      -> data: {"token": "I"} {"token": " found"} {"token": " a slot..."}
```

```
event: completed  -> data: {"message": "Meeting successfully scheduled with Google Meet link attached."}
event: error      -> data: {"error": "Authentication token expired. Please re-authenticate."}
```

4. End-to-End Application Workflow

Step 1: User Sign-In & Outbound Google Permission

The user authenticates on `/sign-in` via Descope (Passkeys/Google). Upon initial login, they click **"Connect Google Calendar"** in the sidebar. Descope executes an Outbound OAuth flow requesting `https://www.googleapis.com/auth/calendar`. The resulting refresh/access tokens are stored securely in Descope's Outbound Vault, returning a verified connection state to Muhurat AI.

Step 2: Conversational Prompt & Multi-Step Reasoning

The user inputs: *"Set a 30-min strategy sync with alex@company.com tomorrow at 11 AM"*. The request streams to `POST /api/agent/stream`. Mastra AI initializes the Gemini agent loaded with system instructions, user working memory, and calendar tools.

Step 3: Conflict Check & Deterministic Google Cloud Mutation

The agent first calls `checkCalendarBusyTool` to verify 11:00 AM – 11:30 AM is open. Upon confirming availability, it executes `createMeetingTool`. The backend resolves the user's Google OAuth token via Descope, calls Google Calendar API v3, attaches Google Meet video conference metadata, and receives a confirmed event ID and Meet URL.

Step 4: Stream Completion & Reactive Visual Calendar Sync

The agent streams the final markdown response with direct Meet and Calendar links. Concurrently, a custom state trigger (`calendarRefreshTrigger`) fires in the frontend, causing the **Month/Week/Day Visual Calendar** to immediately re-fetch the user's primary calendar and display the new meeting block in real-time.

5. Comprehensive FAANG / Senior AI Engineer Interview Bank

CATEGORY A: SYSTEM DESIGN & DISTRIBUTED ARCHITECTURE **5 Questions**

Q1 How did you design the authentication and third-party OAuth token architecture, and why did you decouple them?

Answer: In traditional monolithic applications, developers store Google OAuth access and refresh tokens directly in their application database. This introduces severe security vulnerabilities (plaintext token theft, database leaks) and maintenance overhead (manual token refreshing, handling expiry).

In Muhurat AI, I decoupled user authentication from third-party OAuth tokens using **Descope Outbound Applications**. Descope issues a signed JWT session token to our frontend. When our backend needs to perform calendar mutations on behalf of the user, it makes an authenticated server-to-server call to Descope's Outbound Vault using our backend management key. Descope automatically rotates expired Google tokens and provides a valid temporary access token. This ensures zero third-party credentials ever reside in our PostgreSQL database.

Q2 Why did you choose Server-Sent Events (SSE) instead of WebSockets for the agent chat streaming pipeline?

Answer: WebSockets provide full-duplex communication over a persistent TCP connection, which introduces unnecessary complexity (maintaining stateful connection tables, heartbeat ping/pongs, load balancer sticky sessions) for an AI conversational interface where the communication is unidirectional during generation (Client sends 1 message, Server streams N tokens back).

SSE (Server-Sent Events) operates over standard HTTP/2, leverages native browser `EventSource` or fetch readable streams, is fully stateless, passes cleanly through CDNs and firewalls, supports automatic reconnection, and integrates natively with Next.js edge and Express response streaming without socket infrastructure overhead.

Q3 How does the Model Context Protocol (MCP) server integration work, and what advantages does it offer?

Answer: Model Context Protocol (MCP) is an open standard created by Anthropic that standardizes how AI applications expose context, tools, and prompts to external LLMs and IDEs. We mounted an authenticated MCP server at `POST /mcp` using `@descope/mcp-express`.

This allows external developer environments (such as Antigravity IDE, Cursor, Claude Desktop, or custom CLI agents) to discover and execute Muhurat AI tools (like `createMeeting` and `checkCalendarBusy`) directly within the developer's coding workflow, turning our web app into an extensible scheduling infrastructure.

Q4 **How would you scale this architecture to handle 1,000,000 daily active users with 10,000 concurrent streaming queries?**

Answer:

1. **Stateless Backend Clustering:** Containerize the Express backend with Docker and deploy across Kubernetes / AWS ECS with horizontal auto-scaling based on CPU and open connection count.
2. **Database Read Replicas & Connection Pooling:** Implement PgBouncer / Neon connection poolers with read-replicas for querying user threads and read-heavy calendar views.
3. **Redis Caching for Free/Busy Slots:** Implement an in-memory Redis cache with short TTL (e.g. 60 seconds) for external Google Free/Busy queries to reduce third-party API rate limits.
4. **Asynchronous Task Queue (BullMQ):** Offload background email notifications, recurring calendar audits, and long-running sync jobs to Redis-backed worker queues.
5. **Edge Caching for Static Frontend:** Host Next.js on Vercel Edge Network with static asset caching and stale-while-revalidate for calendar shells.

Q5 **How do you handle race conditions when two users attempt to book the exact same calendar slot simultaneously?**

Answer: We employ a two-layer concurrency control model:

1. **Optimistic Pre-Validation:** The agent checks the Google Calendar Free/Busy API right before constructing the creation payload.
2. **Deterministic Google Cloud Idempotency & Conflict Check:** When committing the event via Google Calendar API v3, we verify against the latest calendar state. In high-concurrency environments, we can implement distributed locking using Redis (Redlock algorithm) keyed on `user_id:slot_timestamp` during the 2-second booking transaction. If the slot is taken midway, the agent catches the conflict exception and automatically proposes the next best available slot to the user.

CATEGORY B: AGENTIC AI & LLM ENGINEERING — 5 Questions

Q6 Why did you choose Google Gemini 3.6 Flash and Mastra AI over frameworks like LangChain or AutoGen?

Answer:

1. **Gemini 3.6 Flash:** Offers state-of-the-art sub-second Time-to-First-Token (TTFT), a 1M+ token context window, precise JSON schema tool execution, and significantly lower latency and cost than heavier legacy models.
2. **Mastra AI Framework:** LangChain is notorious for heavy abstraction bloat, unpredictable breaking changes, and opaque debugging. Mastra provides lightweight, modular TypeScript-first agent primitives, strict Zod schema validation, explicit tool execution loops, and native persistent working memory without unnecessary dependencies.

Q7 How does Mastra Working Memory differ from traditional Vector RAG or simple sliding window buffers?

Answer: Traditional sliding window memory simply passes the last *N* chat messages into the prompt, which quickly exhausts tokens and loses critical user preferences once they scroll past the window. Vector RAG calculates embeddings across historical chunks, but lacks structured temporal understanding.

Mastra Working Memory operates as an active, structured cognitive scratchpad. It continuously synthesizes and updates high-level user traits, working hours (e.g. *"Prefers 25-minute meetings"*, *"Never books on Friday afternoons"*, *"Timezone: IST"*) into a persistent database store. In every new conversation thread, this compact structured profile is injected into the agent's system instructions, providing cross-thread personalization without inflating token costs.

Q8 How do you prevent LLM hallucinations when scheduling dates and times (e.g., relative dates like 'next Tuesday')?

Answer:

1. **Dynamic Temporal Grounding:** In the agent's system prompt instructions (`agent-instructions.ts`), we dynamically inject the exact current server timestamp, day of week, timezone, and calendar date into every invocation.
2. **Strict Zod Date Validation:** Tools require ISO 8601 strings (`YYYY-MM-DDTHH:mm:ssZ`). If Gemini generates an invalid date format, Zod validation fails before any API execution, triggering an automated correction loop.
3. **Deterministic API Enforcement:** The model never "invents" calendar events—it only receives ground truth from the Google Calendar API output.

Q9 How is multi-provider fallback implemented if the primary Gemini API experiences rate limits or outages?

Answer: In `agent.service.ts`, we implemented an automated provider resolution factory. It inspects environment variables and runtime health. If the primary provider (Google Gemini) returns a 429 (Rate Limit) or 503 (Service Unavailable), the error boundary catches the exception and dynamically initializes a secondary fallback model (via OpenRouter or OpenAI) using the identical Mastra tool definitions and system instructions, ensuring zero downtime for end users.

Q10 How do you evaluate and benchmark the reliability of the calendar agent's tool execution?

Answer: We evaluate the agent across 3 core dimensions:

1. **Tool Call Accuracy (Schema Conformance):** Automated unit/integration test suites (e.g. `test-gemini-full.ts`) that run synthetic multi-turn prompts (e.g. ambiguous times, timezone shifts, attendee arrays) and assert that the correct tool was selected with valid Zod-compliant arguments.
2. **End-to-End Latency:** Measuring Time-to-First-Token (TTFT) and Total Execution Time for multi-step tool calls.
3. **Deterministic State Verification:** Verifying that the actual Google Calendar test sandbox matches the database state after agent execution.

CATEGORY C: FRONTEND & FULL-STACK PERFORMANCE — 5 Questions

Q11 How did you architect the dual-theme "Midnight Glass" and "Frosted Pearl" design system?

Answer: Rather than relying on generic CSS utility classes, I built a cohesive design token architecture in `globals.css` leveraging **OKLCH color spaces** and **hardware-accelerated CSS backdrop-filters**:

- **Midnight Glass (Dark Mode):** Deep midnight slate backdrops (`#0b101b`) with 24px backdrop blur, subtle inner specular white borders (`oklch(1 1 1 / 10%)`), glowing cyan gradients, and floating background ambient orbs.
- **Frosted Pearl (Light Mode):** Crisp pearlescent glass cards (`oklch(0.975 0.015 210)`) with 75% opacity, high-contrast typography, and subtle border shadows for maximum daylight legibility.

Theme switching is managed seamlessly via `next-themes` and React Context with zero layout shift or hydration flicker.

Q12 Explain the technical cause and solution for the parent page vertical scrolling bug during chat interactions.

Answer: The Bug: When new chat tokens arrived, `bottomRef.current.scrollIntoView({ behavior: "smooth" })` was invoked. By default, `scrollIntoView()` defaults to `block: "start"` , which causes the browser to shift/scroll *every ancestor scrollable container* in the DOM tree, leading to the entire right dashboard section jumping vertically.

The Solution:

1. Replaced the unconstrained outer wrappers with strict `h-svh overflow-hidden` bounds on the dashboard root.
2. Implemented a dedicated `custom-scrollbar` container with `overflow-y-auto` on the inner message viewport.
3. Switched programmatic scrolling to `scrollContainerRef.current.scrollTo({ top: scrollHeight, behavior: "smooth" })` and `block: "nearest"` , ensuring scrolling was strictly confined to the message area without shifting the header, composer, or calendar view.

Q13 How does the Visual Calendar synchronize reactively in real time when the AI agent schedules a meeting?

Answer: In `chat-panel.tsx` , we maintain a reactive counter state `calendarRefreshTrigger` . When the SSE streaming pipeline emits a `completed` event indicating that the agent has finished a creation, reschedule, or cancellation mutation, `setCalendarRefreshTrigger(prev => prev + 1)` is triggered.

The `CalendarView` component listens to this trigger prop in a `useEffect` hook and automatically re-fetches Google Calendar events for the active date range, ensuring instantaneous UI synchronization between chat actions and the visual calendar without requiring a manual page refresh.

Q14 How is SEO and search engine indexing optimized in a Next.js 16 authenticated web application?

Answer:

1. **Dynamic Metadata & Title Templating:** Configured comprehensive `Metadata` objects in `layout.tsx` with canonical URLs, OpenGraph 1200×1200 preview cards, and Twitter summary cards.
2. **JSON-LD Structured Data:** Embedded a `schema.org` `WebApplication` rich snippet defining Muhurat AI's core capabilities for Google search crawlers.
3. **Programmatic Sitemaps & Robots:** Created Next.js dynamic routes (`sitemap.ts` and `robots.ts`) generating XML sitemaps and indexing policies on-the-fly.
4. **Google Search Console Verification:** Verified domain ownership via Google Search Console HTML meta tag verification.

Q15 What security measures are implemented against XSS, CSRF, and injection attacks in this full-stack app?

Answer:

- **XSS Prevention:** React 19 inherently escapes dynamic string rendering. In `markdown-message.tsx` , markdown is safely parsed with sanitization to prevent arbitrary script execution.
- **CORS & Origin Hardening:** Backend Express CORS whitelist explicitly restricts incoming requests to authorized frontend origins (`APP_URL` , `*.vercel.app`) with strict header allowlists.
- **SQL Injection Immunity:** All PostgreSQL queries in `user.repository.ts` and `connection.repository.ts` utilize parameterized SQL queries (e.g. `$1, $2`) via `pg.Pool` , eliminating SQL injection vectors.
- **Encrypted Token Storage:** Descope Outbound OAuth tokens are AES-256 encrypted at rest and never exposed to the client bundle.

6. Summary & Key Interview Takeaways

What Makes Muhurat AI a Standout Portfolio Project:

- **Production-Grade Agentic Architecture:** It moves beyond simple toy chatbots to demonstrate real-world, deterministic tool calling with external Google Cloud APIs.
- **Decoupled Security Architecture:** Leverages modern identity platforms (Descope) with Outbound Application token vaulting rather than insecure local token storage.
- **Full-Stack Mastery:** Combines Next.js 16, React 19, Tailwind v4, Server-Sent Events, Mastra AI memory, PostgreSQL migrations, and Model Context Protocol (MCP).
- **Aesthetic & UX Excellence:** Features a signature dual-theme glassmorphic interface, interactive visual calendar with split-view, 60fps micro-animations, and seamless responsive design.